SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA0101

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 60 Mins**

No. of Questions:

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I **Yoga Madha A-192425058** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE QUESTION

Intelligent Vehicle Fault Diagnosis System

1. Problem Analysis and Understanding

An automobile service center wants to develop an **AI-based expert system** that can identify possible vehicle faults from symptoms reported by a vehicle. The system uses a knowledge base containing facts and rules about common vehicle problems.

For **Car101**, the system receives the following observations:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

The system uses these observations with predefined rules to identify possible faults and decide whether the vehicle requires immediate service.

The main entities are **Vehicle** and **Fault**. The important predicates are DoesNotStart(x), DimHeadlights(x), HighTemperature(x), LowCoolant(x), BatteryProblem(x), FuelProblem(x), Overheating(x), and NeedsService(x).

The expected inference is that the combination of **not starting** and **dim headlights** indicates a **battery problem**. The system can identify overheating and recommend immediate service only when the required condition of **low coolant** is also known.

2. Knowledge Representation

The knowledge representation consists of:

Entity

Car101

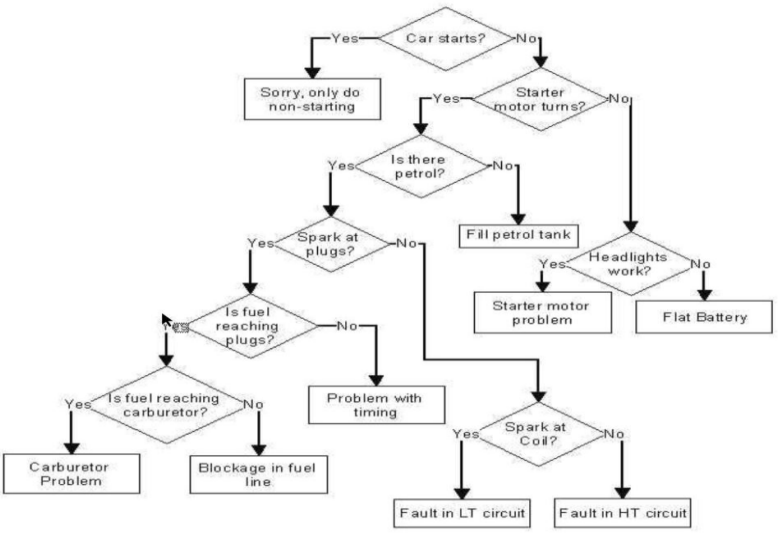
Observed Facts

- DoesNotStart(Car101)
- DimHeadlights(Car101)
- HighTemperature(Car101)

Possible Conclusions

- BatteryProblem(Car101)
- FuelProblem(Car101)
- Overheating(Car101)
- NeedsService(Car101)

Knowledge Flow



3. Knowledge Base

A knowledge base contains two main components:

Knowledge Base = Facts + Rules

3.1 Facts

The facts supplied by the system are:

F1: DoesNotStart(Car101)

F2: DimHeadlights(Car101)

F3: HighTemperature(Car101)

For demonstrating the overheating and service rule completely, an additional diagnostic fact can be considered:

F4: LowCoolant(Car101)

This additional fact is **not present in the original question**. Therefore, without F4, the system cannot logically conclude overheating.

3.2 Rules

Rule 1 – Battery Problem

$DoesNotStart(x) \land DimHeadlights(x) \rightarrow BatteryProblem(x)$

Meaning:

If a vehicle does not start **and** has dim headlights, then it has a battery problem.

Rule 2 – Fuel Problem

$EngineCranks(x) \land DoesNotStart(x) \rightarrow FuelProblem(x)$

Rule 3 – Overheating

$HighTemperature(x) \land LowCoolant(x) \rightarrow Overheating(x)$

Rule 4 – Immediate Service

$Overheating(x) \rightarrow NeedsService(x)$

Rule 5 – Battery Inspection

$BatteryProblem(x) \rightarrow InspectBattery(x)$

Thus, the knowledge base can be represented as:

KB = {F1, F2, F3, R1, R2, R3, R4, R5}

4. First-Order Logic Representation

First-Order Logic represents objects, properties, relationships, and rules using predicates, variables, constants, quantifiers, and logical connectives.

Facts

DoesNotStart(Car101)

DimHeadlights(Car101)

HighTemperature(Car101)

If low coolant is additionally observed:

LowCoolant(Car101)

Rules

Rule 1:

$\forall x\ [DoesNotStart(x) \land DimHeadlights(x) \rightarrow BatteryProblem(x)]$

Rule 2:

$\forall x\ [EngineCranks(x) \land DoesNotStart(x) \rightarrow FuelProblem(x)]$

Rule 3:

$\forall x\ [HighTemperature(x) \land LowCoolant(x) \rightarrow Overheating(x)]$

Rule 4:

$\forall x\ [Overheating(x) \rightarrow NeedsService(x)]$

Rule 5:

$\forall x\ [BatteryProblem(x) \rightarrow InspectBattery(x)]$

Here:

- x = variable representing any vehicle.
- Car101 = constant.
- ∧ = AND.
- → = implies.
- ∀ = for all.

5. Propositional Logic Representation

For Propositional Logic, each statement is treated as a proposition.

Let:

P = Car101 does not start

Q = Car101 has dim headlights

R = Car101 has high temperature

S = Car101 has low coolant

T = Car101 has a battery problem

U = Car101 has a fuel problem

V = Car101 has an overheating problem

W = Car101 needs immediate service

Facts

P

Q

R

If low coolant is known:

S

Rules

Battery rule:

$P \land Q \rightarrow T$

Fuel rule:

$EngineCranks \land P \rightarrow U$

Overheating rule:

$R \land S \rightarrow V$

Service rule:

$V \rightarrow W$

Therefore:

$$P \land Q$$
$$\downarrow$$
$$T$$

So:

BatteryProblem(Car101)

If S is also known:

$$R \land S$$
$$\downarrow$$
$$V$$
$$\downarrow$$
$$W$$

Therefore:

$$Overheating(Car101)$$
$$\downarrow$$
$$NeedsService(Car101)$$

6. Unification

Unification is the process of finding substitutions that make two logical expressions identical.

Consider the rule:

DoesNotStart(x) ∧ DimHeadlights(x) → BatteryProblem(x)

and the fact:

DoesNotStart(Car101)

We compare:

DoesNotStart(x)

DoesNotStart(Car101)

The required substitution is:

{x / Car101}

Therefore:

DoesNotStart(x)
↓ {x / Car101}
DoesNotStart(Car101)

The expressions are now identical.

Unification Example 2

Consider:

HighTemperature(x)

and:

HighTemperature(Car101)

The substitution is:

{x / Car101}

Therefore:

HighTemperature(x)
↓ {x / Car101}
HighTemperature(Car101)

MGU

The **Most General Unifier (MGU)** is:

MGU = {x / Car101}

It allows the general rule to be applied specifically to Car101.

7. Forward Chaining

Forward chaining is a data-driven reasoning method. It starts with known facts and repeatedly applies rules to generate new facts until the required conclusion is obtained.

Initial Facts

F1 = DoesNotStart(Car101)

F2 = DimHeadlights(Car101)

F3 = HighTemperature(Car101)

Step 1

Apply Rule 1:

DoesNotStart(x) ∧ DimHeadlights(x)
→ BatteryProblem(x)

Substitute:

x = Car101

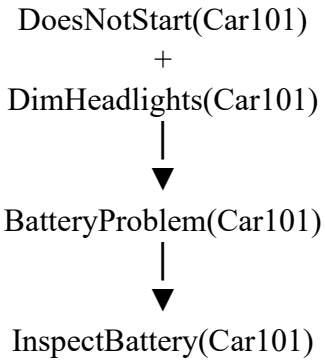
We have:

DoesNotStart(Car101) ∧ DimHeadlights(Car101)

Therefore:

BatteryProblem(Car101)

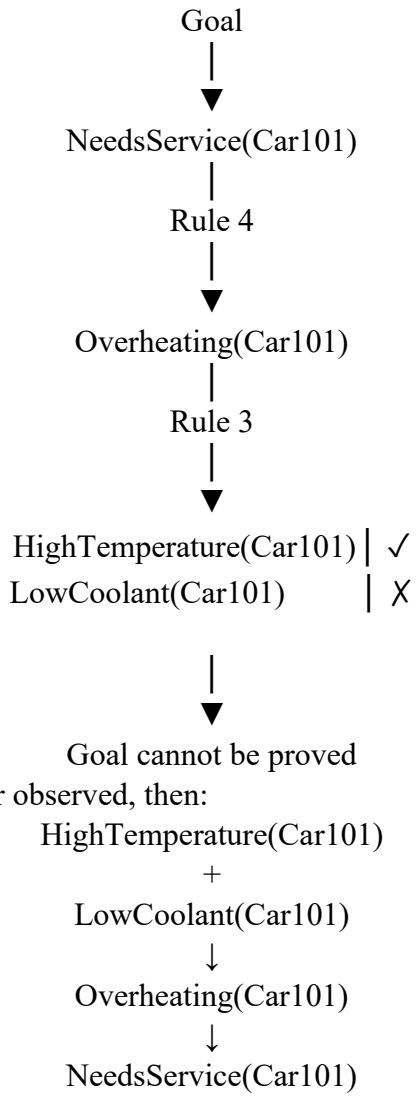
Step 2
Apply Rule 5:
BatteryProblem(x) → InspectBattery(x)
Therefore:
InspectBattery(Car101)
Forward-Chaining Diagram



Result
The system identifies:
Car101 has a possible battery problem.
For overheating:
HighTemperature(Car101)
alone is **not sufficient** to trigger:
Overheating(Car101)
because Rule 3 also requires:
LowCoolant(Car101)
Therefore, the logically correct conclusion from the original facts is:
BatteryProblem(Car101)

8. Backward Chaining for NeedsService(Car101)
Backward chaining is **goal-driven reasoning**. It starts with a goal and works backward to determine which facts are required to prove that goal.
Query
NeedsService(Car101)
Look for a rule that concludes NeedsService(x).
We find:
Overheating(x) → NeedsService(x)
Therefore, to prove:
NeedsService(Car101)
we need:
Overheating(Car101)
Now look for a rule that concludes Overheating(x):
HighTemperature(x) ∧ LowCoolant(x)
→ Overheating(x)
Therefore, we need:
HighTemperature(Car101)
and:
LowCoolant(Car101)
We have:
HighTemperature(Car101) ✓
LowCoolant(Car101) ✗
The original problem does **not** provide LowCoolant(Car101).

Therefore:
NeedsService(Car101)
cannot be proved from the original knowledge base.
Backward-Chaining Diagram

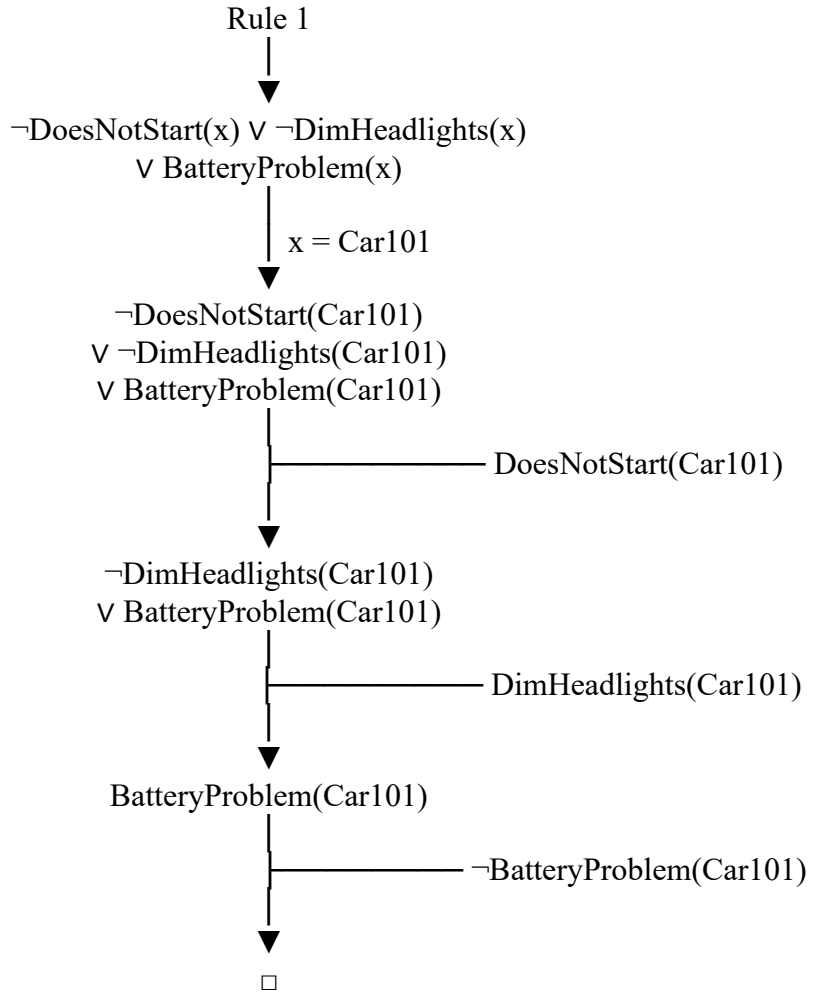


If LowCoolant(Car101) is later observed, then:

9. Resolution
Resolution is an inference technique used to prove whether a query logically follows from a knowledge base.
We select the query:
BatteryProblem(Car101)
The relevant rule is:
DoesNotStart(x) ∧ DimHeadlights(x)
→ BatteryProblem(x)
Convert the implication into clause form:
¬DoesNotStart(x) ∨ ¬DimHeadlights(x) ∨ BatteryProblem(x)
Substitute:
x = Car101
Therefore:
¬DoesNotStart(Car101)
∨ ¬DimHeadlights(Car101)
∨ BatteryProblem(Car101)
Known facts are:
DoesNotStart(Car101)

DimHeadlights(Car101)
To prove the query using resolution, negate the query:
¬BatteryProblem(Car101)
Now resolve:
¬DoesNotStart(Car101)
∨ ¬DimHeadlights(Car101)
∨ BatteryProblem(Car101)
with:
DoesNotStart(Car101)
This gives:
¬DimHeadlights(Car101)
∨ BatteryProblem(Car101)
Resolve with:
DimHeadlights(Car101)
We obtain:
BatteryProblem(Car101)
Now resolve with the negated query:
¬BatteryProblem(Car101)
This produces:
□
where □ represents the **empty clause / contradiction**.
Therefore, the negated query is false and the original query is true:
∴ BatteryProblem(Car101)

10. Resolution Diagram

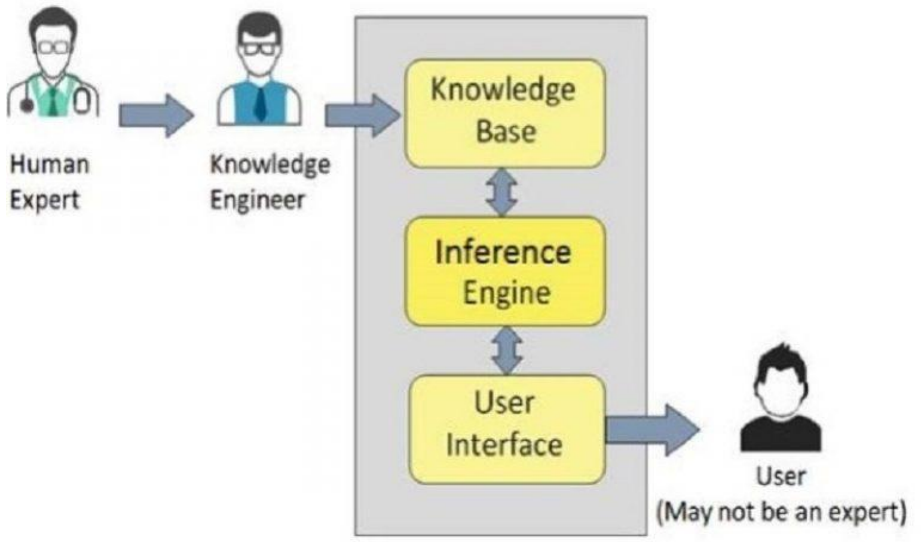


CONTRADICTION

↓

∴ BatteryProblem(Car101)

11. Knowledge Representation Diagram / System Architecture



12. Complete Inference Process

The complete reasoning process for Car101 is:

FACTS

- DoesNotStart(Car101)
- DimHeadlights(Car101)
- HighTemperature(Car101)

↓

RULE 1

DoesNotStart ∧ DimHeadlights

↓

BatteryProblem(Car101)

↓

InspectBattery(Car101)

For the service decision:

HighTemperature(Car101)

+

LowCoolant(Car101)

↓

Overheating(Car101)

↓

NeedsService(Car101)

However, because **LowCoolant(Car101) is not among the original observations**, the system must not automatically infer Overheating(Car101) or NeedsService(Car101).

13. Final Inference

The knowledge-based system successfully analyzes the symptoms of **Car101**.
From:
DoesNotStart(Car101)
and:
DimHeadlights(Car101)
the system applies the rule:
DoesNotStart(x) ∧ DimHeadlights(x)
→ BatteryProblem(x)
and derives:
BatteryProblem(Car101)
Therefore, the primary diagnosis supported by the given facts is:
Car101 has a possible battery problem and the battery should be inspected.
Although Car101 has a high temperature, the system cannot conclude that the vehicle is overheating because the rule requires both:
HighTemperature(Car101)
and:
LowCoolant(Car101)
Since LowCoolant(Car101) is not given, the query:
NeedsService(Car101)
cannot be logically proved from the original knowledge base.

14. Conclusion

The intelligent vehicle fault diagnosis system demonstrates how a knowledge-based system can use **facts, rules, First-Order Logic, Propositional Logic, unification, forward chaining, backward chaining, and resolution** to reason about vehicle problems. Forward chaining derives the battery problem directly from the observed symptoms, while backward chaining shows that the service requirement depends on proving an overheating condition. Resolution also confirms the battery diagnosis logically. This demonstrates how an inference engine can convert raw vehicle symptoms into meaningful diagnostic conclusions while avoiding unsupported conclusions.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.